

Lab 2 – Reflection

Guru R Bharadwaj | SRN: PES1UG24CS177 | Section: 5-C

Problem Statement #48: Incident Escalation & On-Call Rotation Engine — Jira project: OCE

1. Did your estimations reflect the actual effort?

Not evenly. Story 3.3 (Halt Escalation on Acknowledgement) was the clearest miss: it carried an 8-point estimate reflecting the number of subsystems it touches – the acknowledgement path, the notification dispatcher, and the audit trail all have to agree an incident is now owned – and it was the one story that did not land inside Sprint 1's window, carrying into Sprint 2 where it was the first thing completed. That is a reasonable estimate proven right by the outcome: it genuinely took longer than a same-sized scheduling story.

By contrast, the scheduling stories in Epic 2 (3, 3, 5, 5 points) all landed inside Sprint 1 without incident, and the three Epic 1 stories picked up in Sprint 2 (5, 3, 8 points) cleared inside a day and a half. The pattern that emerges: stories whose uncertainty was about *coordination between subsystems* (3.3, and to a lesser extent the escalation-ladder stories) were harder to size correctly than stories whose uncertainty was about *scope of a single feature* (the scheduling stories). Fibonacci's growing gaps at the high end (5, 8) are meant to carry exactly this kind of uncertainty, and here they did their job – the estimate was rough but the direction (higher effort, higher risk of slipping) was right.

2. Was your backlog well-prioritized?

Yes, and the ordering held up under an actual dependency the priority scores did not explicitly capture. Epic 1 (Alert Ingestion & Escalation Engine) is the headline feature and carries the two Highest-priority stories in the whole backlog, yet it was deliberately scheduled *second*: the escalation engine cannot resolve who to page until the rotation schedule (Epic 2) and the acknowledgement path (Epic 3) exist. Business value alone would have put Epic 1 first; the dependency graph overruled that, and the sprints delivered cleanly as a result – nothing in Sprint 2 was blocked waiting on Sprint 1 work that had not shipped.

Post-mortem work (Epic 4) sat at Medium/Low priority throughout and never got scheduled, which is correct: an incident is survivable without a post-mortem, so it belongs behind the escalation and acknowledgement paths that determine whether an incident gets handled at all.

3. How did your simulated sprint align with your plan?

Sprint 1 committed 32 points across 7 stories and delivered 24 – Story 3.3 (8 points) missed the window and carried over, which is exactly the kind of real result the plan anticipated by committing to full-epic scope rather than an artificially safe number. Sprint 2 then committed 24 points, set directly from Sprint 1's measured velocity rather than repeating the 32-point assumption, and delivered all 24 with no carry-over.

That progression – a first sprint that slightly over-commits and re-plans off real velocity, followed by a second sprint that lands exactly on forecast – is a more honest simulation of sprint planning than two sprints that both hit 100% of an identical, pre-chosen number. The re-plan between sprints is the mechanism that makes velocity useful at all, and this is the first point in the exercise where that mechanism actually did something.

4. What insights did the burndown chart give about your team's capacity?

Sprint 1's burndown (32 to 24 to 21 to 8, run across a real Saturday-to-Sunday window and only resolved by explicitly toggling “Show Non-Working Days” so the chart wouldn't compress the weekend into an unreadable sliver) never reached zero, which is the correct signal that the sprint over-committed relative to the two days available to it. Sprint 2's burndown (24 to 16 to 8 to 0, run across Monday-to-Tuesday) reached zero cleanly with the guideline tracked closely throughout – a much better match between commitment and capacity.

Reading the two charts side by side tells the capacity story better than either one alone: Sprint 1 establishes that ~24 points is closer to this team's real two-day throughput than the 32 points originally committed, and Sprint 2 confirms it by hitting that revised number exactly. One sprint is a guess; two sprints, with a deliberate re-plan between them, is the beginning of an actual velocity figure – which is the entire point of tracking it.

A smaller but concrete lesson: the burndown chart itself needs “Show Non-Working Days” enabled whenever a sprint's timebox includes a weekend, or Jira's default rendering silently truncates and mislabels the axis. That is a tooling detail rather than a team-capacity one, but it is the kind of thing that would have produced a misleading chart in a real retrospective if not caught.